

http://algs4.cs.princeton.edu

1.4 ANALYSIS OF ALGORITHMS

- *introduction*
- *observations*
- *mathematical models*
- *order-of-growth classifications*
- *theory of algorithms*
- *memory*



1.4 Analysis of Algorithms

- *introduction*
- *observations*
- *mathematical models*
- *order-of-growth classifications*
- *theory of algorithms*
- *memory*

Arsyet për të analizuar algoritmet

Parashikimi i performancës.

Krahasimi i algorithms.

Garantimi i korrektësisë

Bazat teorike të algoritmeve.

Arsyeja kryesorë praktike:

Tejkalimi i performance bugs.



klienti ka performance të dobët sepse programeri nuk i ka kuptuar karakteristikat e performancës



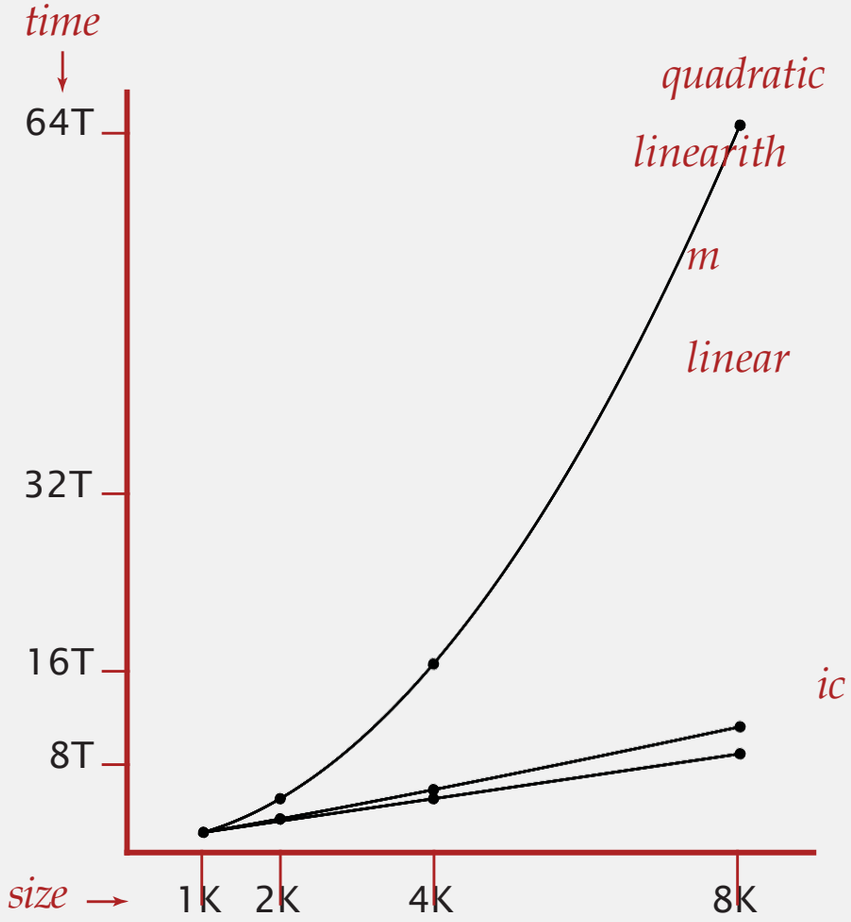
Disa algoritme të suksesshme

Transformimet diskrete Fourier.

- Konvertimi i formave valore me N mostra në komponenta periodike
- Aplikimi: DVD, JPEG, MRI, astrofiyzica, ….
- Brute force: N^2 hapa.
- FFT algoritmi: $N \log N$ hapa, mundëson teknologji të re.



Friedrich Gauss
1805





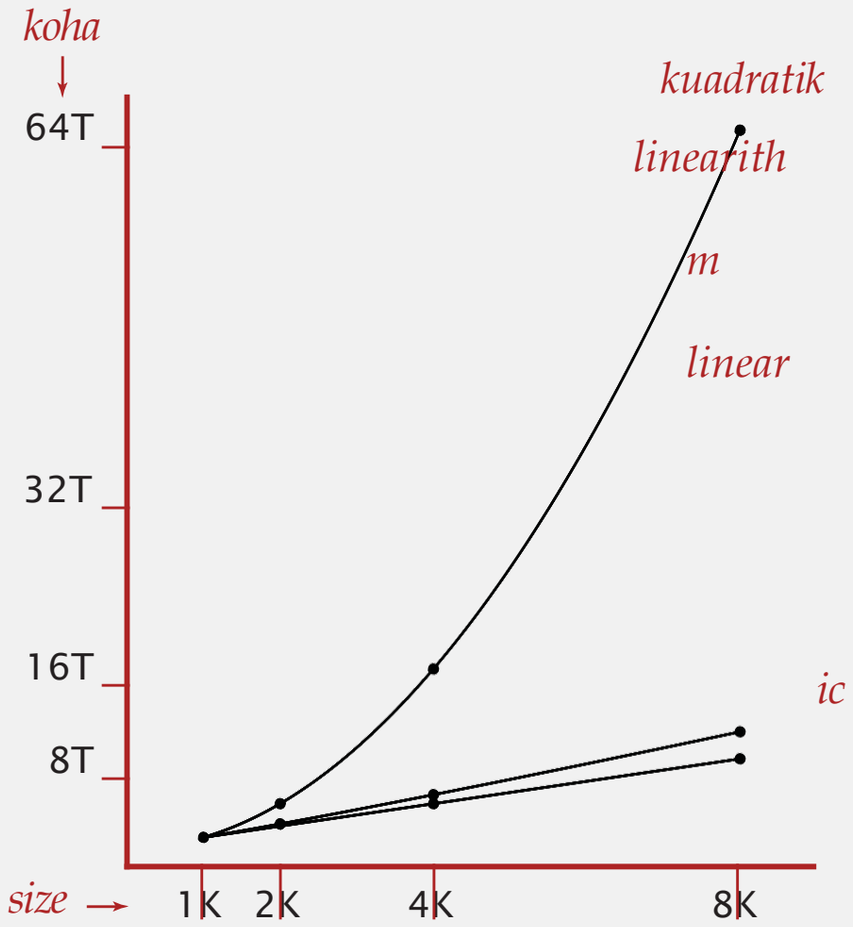
Disa algoritme të suksesshme

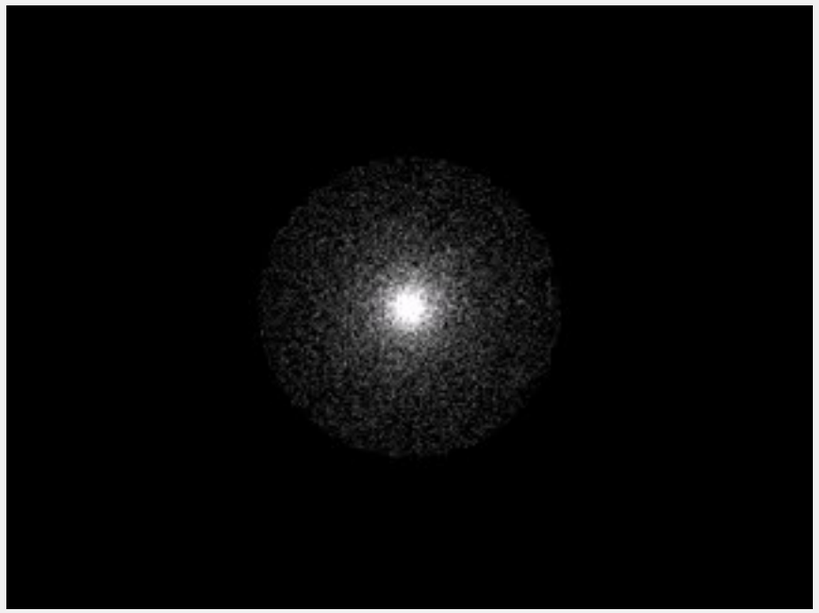
N-body simulimet.

- Simulohen bashkëveprimet gravitacionale ndëmjet N trupave.
- Brute force: N^2 hapa.
- Barnes-Hut algoritmi: $N \log N$ hapa, mundëson hulumtim të ri.



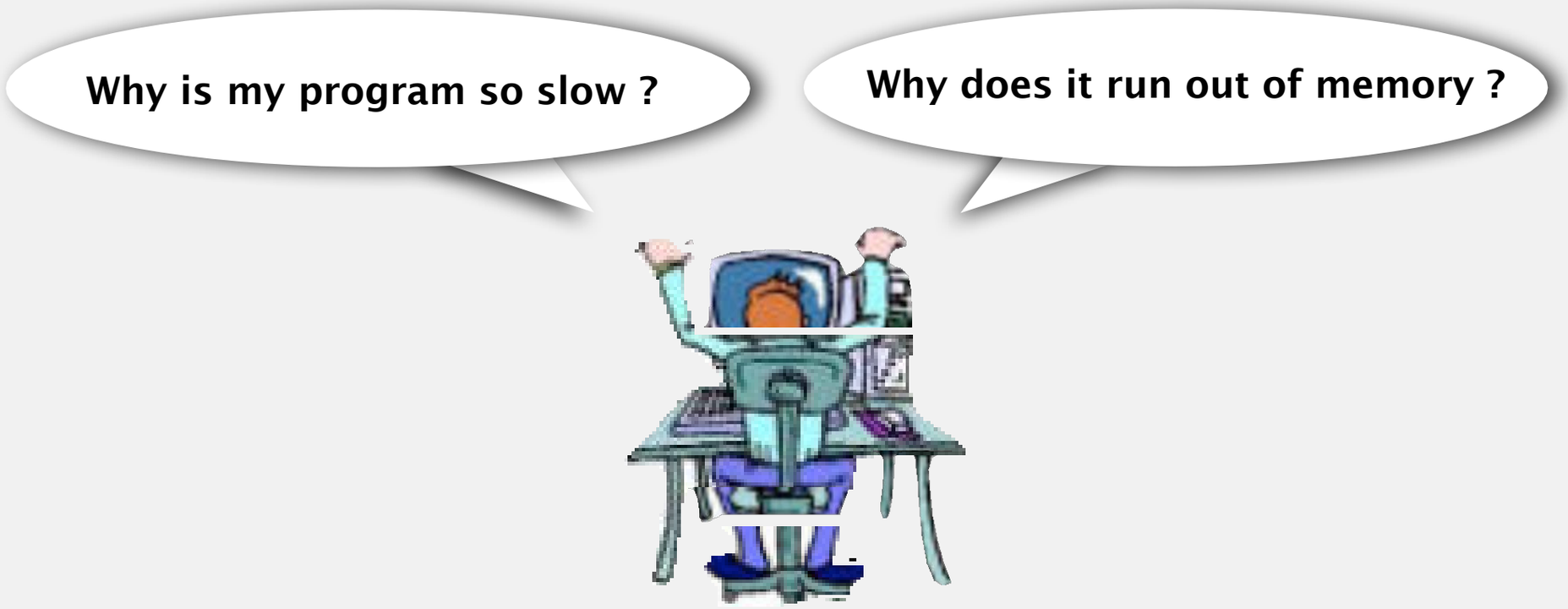
Andrew Appel
PU '81





Sfida

Q. A do të jetë në gjendje programi të zgjidhë problem me input të madh të të dhënave?



[Knuth 1970ve] Përdorimi i metodave shkencore për të kuptuar performancën.

Zbatimi i metodave shkencore në analizën e algoritmeve

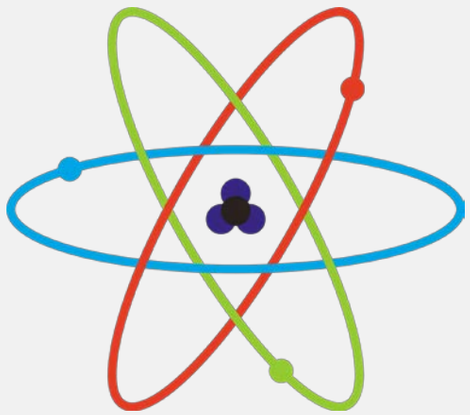
Struktura për parashikimin e performancës dhe krahasimit të algoritmeve.

Metodat shkencore.

- Observimi i ndonjë tipari/fenomeni natyror në botë.
- Hipoteza një model që është në përputhshmëri me observimet.
- Parashikimi i ngjarjes duke përdorë hipotezën.
- Verifikimi i parashikimieve duke bërë observime të rradhës.
- Validimi duke përsëritur derisa hipoteza dhe observimi përputhën.

Parimet.

- Eksperimentet duhet të jenë të rifabrikueshme.
- Hipoteza duhet të jetë e mohueshme.





http://algs4.cs.princeton.edu

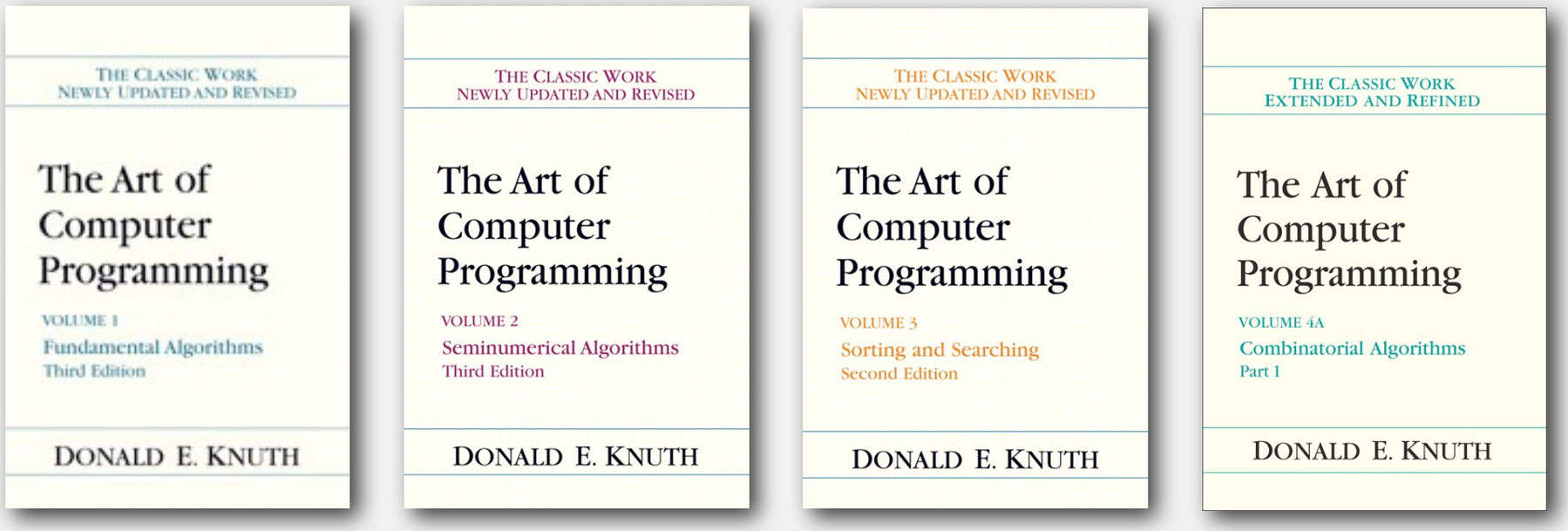
1.4 Analysis of Algorithms

- introduction
- observations
- *mathematical models*
- order-of-growth classifications
- theory of algorithms
- memory

Modeli matematikor për running time

Total running time: shuma e çmimit x frekuenca për të gjitha operacionet.

- Duhet analiza e programit për të përcaltuar bahkësinë e operacioneve.
- Çmimi varet nga maqina, kompajlleri.
- Frekuenca varet nga algoritmi, input-I i të dhënave.





Donald Knuth
1974 Turing Award

Në parim, ekzistojnë modele matematikoretë sakta.

Çmimi i operacioneve themelore

operation	example	nanoseconds †
integer add	`a + b`	2.1
integer multiply	`a * b`	2.4
integer divide	`a / b`	5.4
floating-point add	`a + b`	4.6
floating-point multiply	`a * b`	4.2
floating-point divide	`a / b`	13.5
sine	`Math.sin(theta)`	91.3
arctangent	`Math.atan2(y, x)`	129.0
...	...	...

† Running OS X on Macbook Pro 2.2GHz with 2GB RAM

Çmimi i operacioneve themelore

Observimi. Shumica e operacioneve primitive marrin kohë konstante.

operation	example	nanoseconds †
variable declaration	`int a`	c_1
assignment statement	`a = b`	c_2
integer compare	`a < b`	c_3
array element access	`a[i]`	c_4
array length	`a.length`	c_5
1D array allocation	`new int[N]`	$c_6 \ N$
2D array allocation	`new int[N][N]`	$c_7 \ N^2$

Shembull: 1-SUM

Q. Sa instrukcione ka funksioni me input N?

```
int count = 0;
for (int i = 0; i < N; i++)
   if (a[i] == 0)
      count++;
```

N array accesses

operation	frequency
variable declaration	2
assignment statement	2
less than compare	$N + 1$
equal to compare	N
array access	N
increment	N to $2\ N$

Shembull:2-SUM

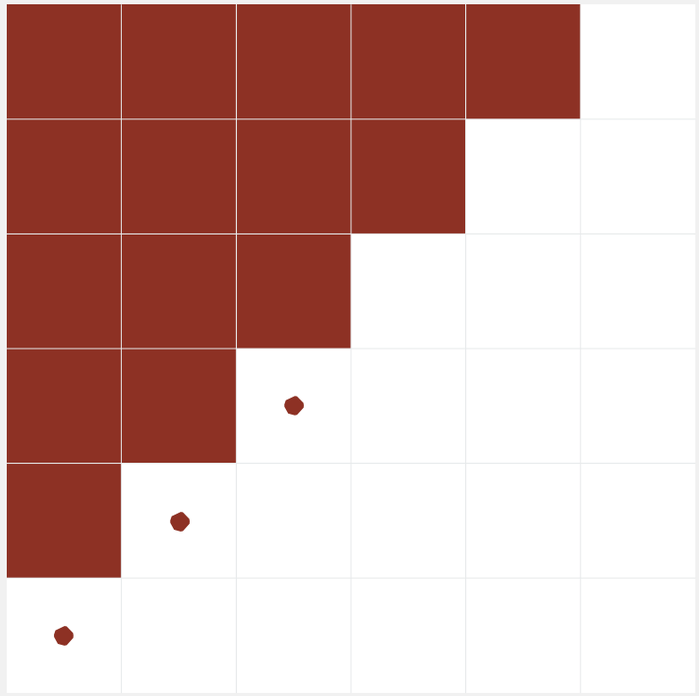
Q. Sa instrukcione ka funksioni me input N?

```
int count = 0;
for (int i = 0; i < N; i++)
   for (int j = i+1; j < N; j++)
      if (a[i] + a[j] == 0)
         count++;
```

$$0 + 1 + 2 + \ldots + (N - 1) = \frac{1}{2} N (N - 1)$$

$$= \frac{N^{\rightarrow 1}}{2}$$

Pf. [n even]



$$0 + 1 + 2 + \ldots + (N - 1) = \frac{1}{2} N^2 - \frac{1}{2} N$$

half of square

half of diagonal

Shembull:2-SUM

Q. Sa instrukcione ka funksioni me input N?

```
int count = 0;
for (int i = 0; i < N; i++)
   for (int j = i+1; j < N; j++)
      if (a[i] + a[j] == 0)
         count++;
```

$$0 + 1 + 2 + \ldots + (N - 1) = \frac{1}{2} N (N - 1) = \frac{N^2}{2}$$

operation	frequency
variable declaration	$N + 2$
assignment statement	$N + 2$
less than compare	$\frac{1}{2} (N + 1) (N + 2)$
equal to compare	$\frac{1}{2} N (N - 1)$
array access	$N (N - 1)$
increment	$\frac{1}{2} N (N - 1)$ to $N (N - 1)$

tedious to count exactly

Shembull:3-SUM

Q. Përafësisht sa array qasje si funksion me input N?

```
int count = 0;
for (int i = 0; i < N; i++)
   for (int j = i+1; j < N; j++)
      for (int k = j+1; k < N; k++)
         if (a[i] + a[j] + a[k] == 0)
            count++;
```

"inner loop"

$$\binom{N}{3} = \frac{N(N-1)(N-2)}{3!} \rightarrow \frac{1}{6}N^3$$

A. $\sim \frac{1}{2} N^3$ array qasje.



http://algs4.cs.princeton.edu

1.4 Analysis of Algorithms

- introduction
- observations
- mathematical models
- *order-of-growth classifications*
- theory of algorithms
- memory

Shkalla e rritjës së funksionit

Definim. Nëse $f(N) \sim c\, g(N)$ për një konstantë $c > 0$, atëherë order of growth (shkalla e rritjës) e $f(N)$ është $g(N)$.

- Injorohet koeficienti i pare (konstanta).
- Injorohen shkallët (fuqitë) më të ulëta.

P.sh. Order of growth për running time të këtij kodi është N^3.

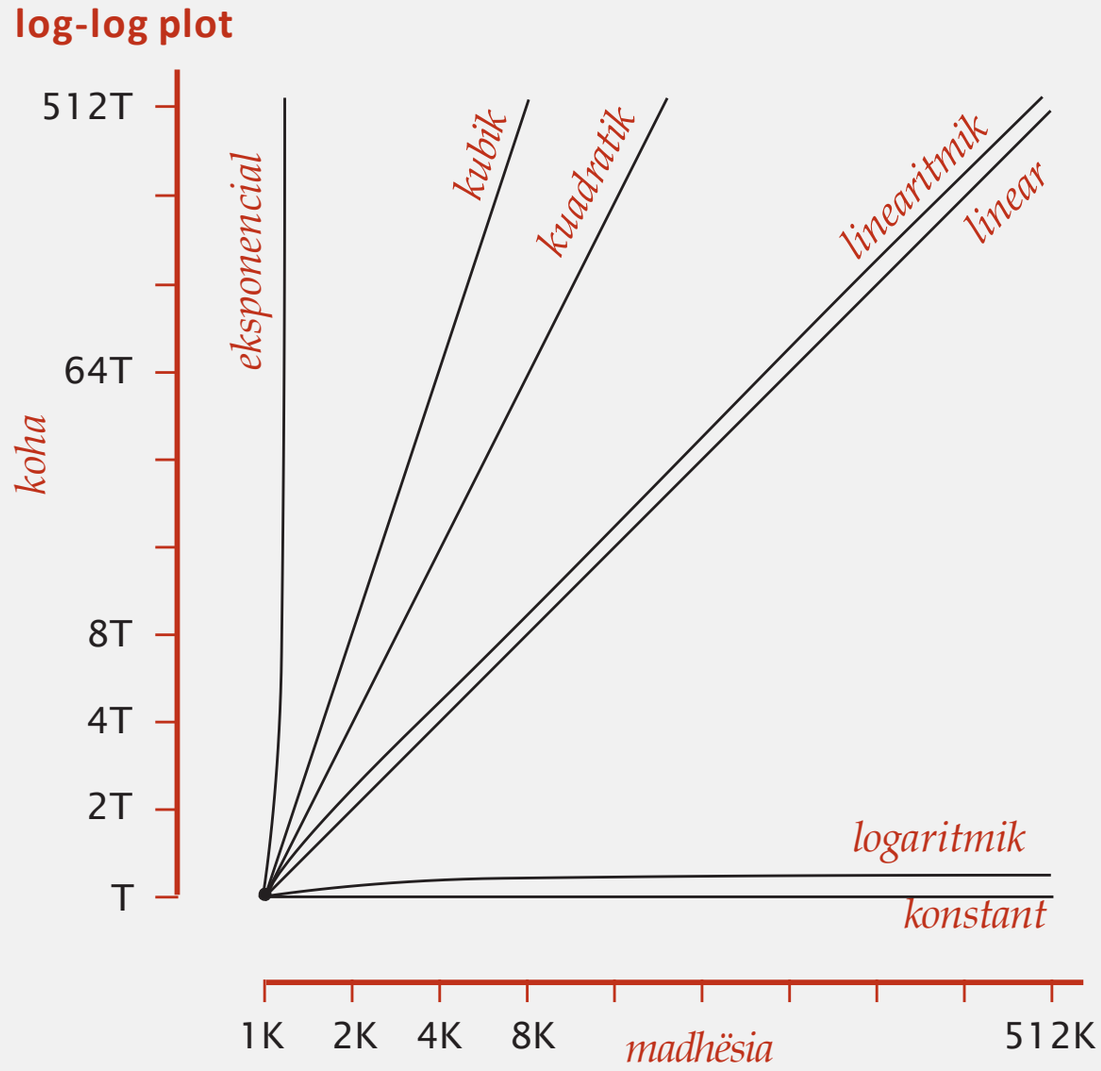
```
int count = 0;
for (int i = 0; i < N; i++)
   for (int j = i+1; j < N; j++)
      for (int k = j+1; k < N; k++)
         if (a[i] + a[j] + a[k] == 0)
            count++;
```

Shkalla e rritjës së funksionit

Përparësi. Bashkësia e funsioneve

$$1,\ \log N,\ \ N,\ \ N \log N,\ \ N^2,\ \ N^3, \text{ dhe } 2^N$$

mjafton për të përshkruar order of growth të shumicës së algoritmeve.



Typical orders of growth

Shkalla e rritjës së funksionit

order of growth	name	typical code framework	description	example	$T(2N) / T(N)$
1	**constant**	`a = b + c;`	statement	add two numbers	1
$\log N$	**logarithmic**	`while (N > 1) { N = N / 2; ... }`	divide in half	binary search	~ 1
N	**linear**	`for (int i = 0; i < N; i++) { ... }`	loop	find the maximum	2
$N \log N$	**linearithmic**	[see mergesort lecture]	divide and conquer	mergesort	~ 2
N^2	**quadratic**	`for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { ... }`	double loop	check all pairs	4
N^3	**cubic**	`for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) for (int k = 0; k < N; k++) { ... }`	triple loop	check all triples	8
2^N	**exponential**	[see combinatorial search lecture]	exhaustive search	check all subsets	$T(N)$

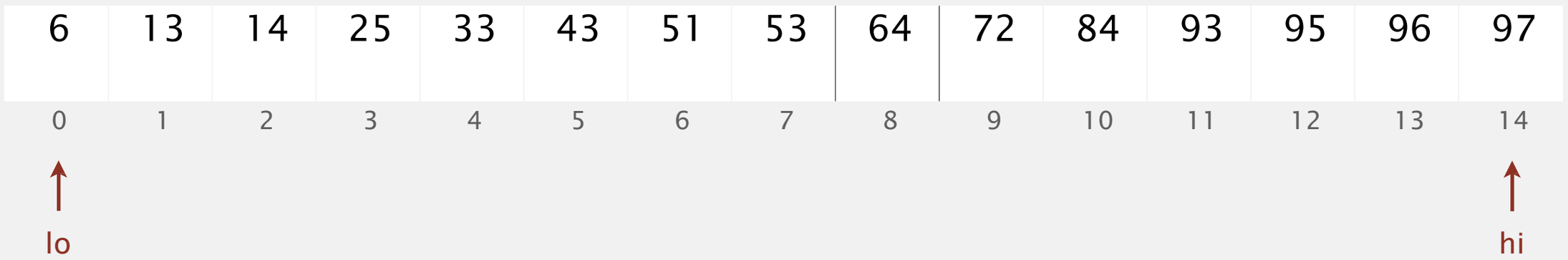
Binary search demo

Qëllimi. Marrë një array të sortuar dhe një çelës, të gjindet index-i i çelësit në atë array?

Binary search. Krahasohet çelësi me vlerën e mesme.

- Më e vogël, shko majtas.
- Më e madhe, shko djathtas.
- Vlera e barabartë, është gjetur rezultati.

Kërkim i suksesshëm për 33



Krahasimi i programeve

Hipotezë. Algoritmi sortues $N^2 \log N$ për 3-SUM është dukshëm më I shpejtë realisht sesa brute-force N^3 algoritmi.

N	time (seconds)
1,000	0.1
2,000	0.8
4,000	6.4
8,000	51.1

ThreeSum.java

N	time (seconds)
1,000	0.14
2,000	0.18
4,000	0.34
8,000	0.96
16,000	3.67
32,000	14.88
64,000	59.16

ThreeSumDeluxe.java

Parimi i punës. Sa më e ulët order of growth, aq më i shpejtë programi



http://algs4.cs.princeton.edu

1.4 Analysis of Algorithms

- *introduction*
- *observations*
- *mathematical models*
- *order-of-growth classifications*
- *theory of algorithms*
- *memory*

Notacionet e përdorura në teorinë e algoritmeve

notation	provides	example	shorthand for	used to
Big Theta	asymptotic order of growth	$\Theta(N^2)$	$\frac{1}{2} N^2$ $10 N^2$ $5 N^2 + 22 N \log N + 3N$ ⋮	classify algorithms
Big Oh	$\Theta(N^2)$ and smaller	$O(N^2)$	$10 N^2$ $100 N$ $22 N \log N + 3 N$ ⋮	develop upper bounds
Big Omega	$\Theta(N^2)$ and larger	$\Omega(N^2)$	$\frac{1}{2} N^2$ N^5 $N^3 + 22 N \log N + 3 N$ ⋮	develop lower bounds



http://algs4.cs.princeton.edu

1.4 Analysis of Algorithms

- *introduction*
- *observations*
- *mathematical models*
- *order-of-growth classifications*
- *theory of algorithms*
- *memory*

Memoria - bazat

Bit. 0 ose 1.

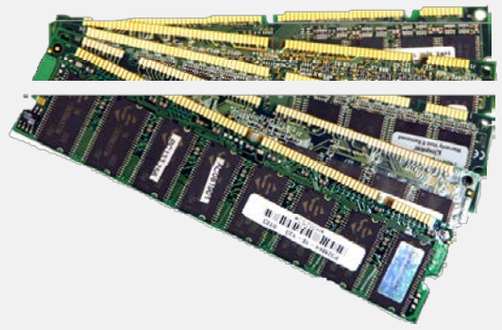
Byte. 8 bita.

Megabyte (MB). 1 million ose 2^{20} bytes.

Gigabyte (GB). 1 billion ose 2^{30} bytes.

NIST

most computer scientists



64-bit maqina. Supozohet një 64-bit maqinë me 8-byte pointera.

- Mund të adreson më shumë memorie.
- Pointerët zënë më shumë hapësirë.

disa JVM "kompresojnë" pointerët e objektet deri në 4 byte për ta ulur çmimin



Shpenzimi i memories tek tipet primitive dhe array-at

type	bytes
boolean	1
byte	1
char	2
int	4
float	4
long	8
double	8

primitive types

type	bytes
char[]	$2N + 24$
int[]	$4N + 24$
double[]	$8N + 24$

one- dimensional arrays

type	bytes
char[][]	$\sim 2MN$
int[][]	$\sim 4MN$
double[][]	$\sim 8MN$

two- dimensional arrays

Shpenzimi i memories tek objektet në Java

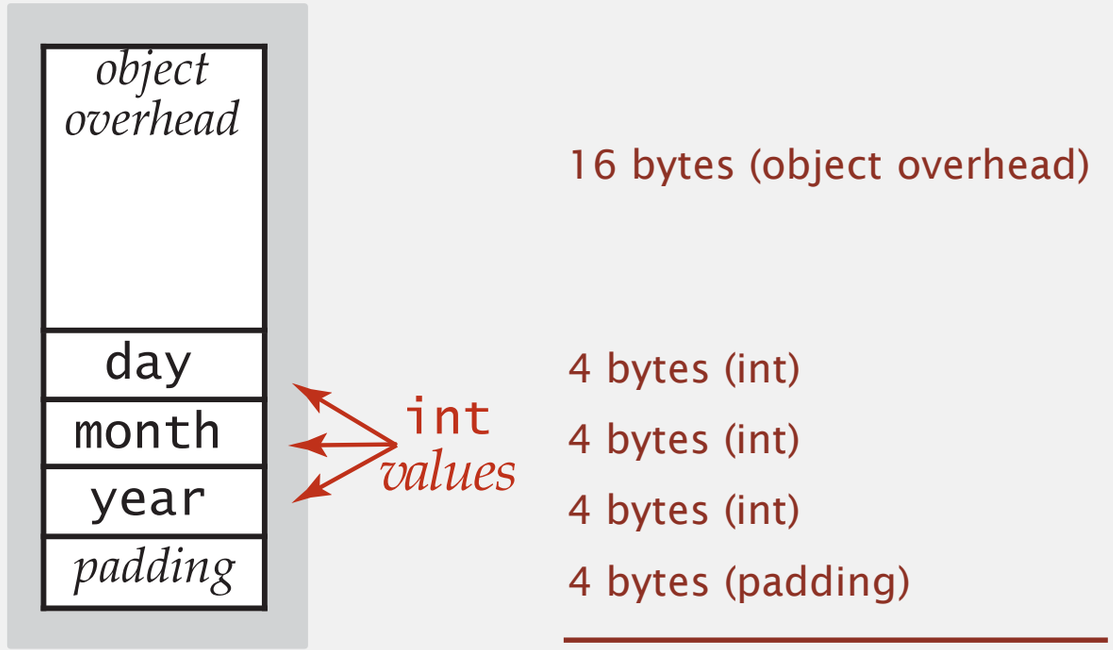
Overhead i objektit. 16 bytes.

Referenca. 8 bytes.

Padding. Çdo objekt përdorë shumëfisha të 8 byte-ave.

P.sh. Një objekt për datën përdorë32 byte memorie.

```
public class Date
{
   private int day;
   private int month;
   private int year;
...
}
```



Shpenzimi i memories - përmbledhje

Përdorimi i memories për një vlerë të caktuar të tipeve të të dhënave:

- Tipe primitive: 4 bytes për `int`, 8 byte për `double`, ...
- Reference e objektit: 8 bytes.
- Array: 24 byte + memoria për çdo array vlerë.
- Objekti: 16 byte + memoria për çdo instance të variablës.
- Padding: Rrumbullakëson deri në shumëfish të 8 byte.



+ 8 extra bytes per inner class object
(for reference to enclosing class)

Shembull

Q. Sa memorie përdorë `WeightedQuickUnionUF` si funksion i N?

```
public class WeightedQuickUnionUF
{
   private int[] id;
   private int[] sz;
   private int count;

   public WeightedQuickUnionUF(int N)
   {
      id = new int[N];
      sz = new int[N];
      for (int i = 0; i < N; i++) id[i] = i;
      for (int i = 0; i < N; i++) sz[i] = 1;
   }
   ...
}
```

16 bytes
(object overhead)

8 + (4N + 24) bytes each
(reference + `int[]` array)

4 bytes (`int`)

4 bytes (padding)

8N + 88 bytes

A. $8N + 88 \sim 8N$ bytes.